Roadmap detallado del proyecto

Mapa de calor con ESP32, ESP-NOW, RSSI y analisis de datos

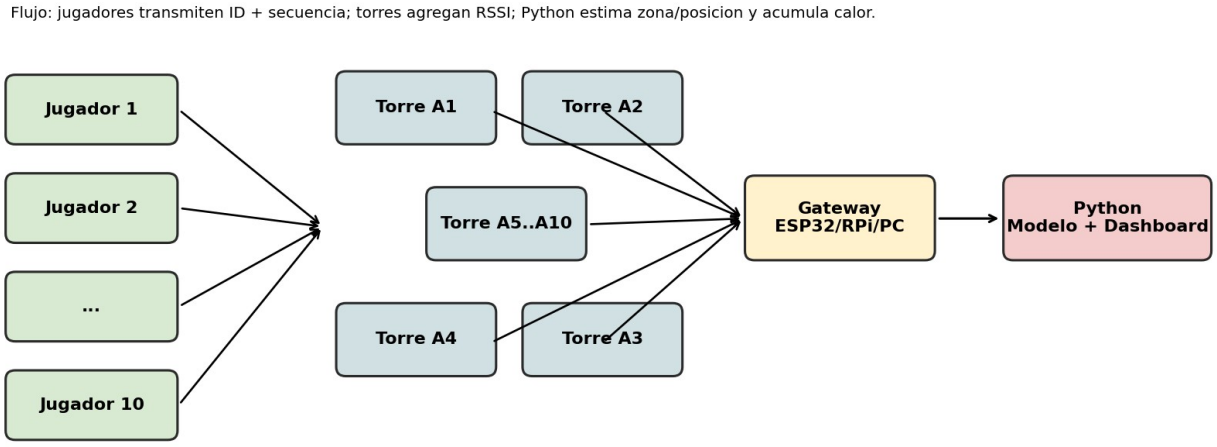


Figura 1. Arquitectura conceptual del sistema.

Proyecto	Sistema de estimacion de presencia y movimiento aproximado por cuadrantes
Tecnologias base	ESP32, ESP-NOW, RSSI, Python, aprendizaje supervisado
Alcance inicial	1 jugador, 4 torres, cuadricula 3x3, dataset CSV, modelo KNN
Escalamiento objetivo	10 jugadores, 8 a 10 torres, 16 o 25 zonas, visualizacion en pantalla
Version del documento	v1.0 - Junio 2026

Indice del documento

1. Resumen ejecutivo
2. Vision y alcance tecnico
3. Arquitectura propuesta
4. Decisiones de diseno
5. Roadmap detallado por fases
6. Cronograma recomendado
7. Dataset, calibracion y modelos
8. Visualizacion, heatmap e interpolacion
9. Escalamiento a 10 jugadores
10. Plan de pruebas y metricas
11. Riesgos principales y mitigaciones
12. Checklist del MVP
13. Glosario y referencias

Idea guia
El proyecto no debe presentarse como posicionamiento exacto tipo GPS. Debe presentarse como un sistema de estimacion de presencia y movimiento aproximado, basado en patrones de intensidad RSSI y validado por cuadrantes.

1. Resumen ejecutivo

El proyecto propone construir un sistema de bajo costo para generar mapas de calor de una zona despejada usando dispositivos ESP32. La idea es que cada jugador porte un ESP32 movil que transmite periodicamente un identificador y un numero de secuencia mediante ESP-NOW. Alrededor del area se colocan varios ESP32 fijos, denominados torres o anchors, que reciben esos paquetes y registran la intensidad de la senal recibida, conocida como RSSI.

Con esos datos, Python reconstruye un vector de intensidades por jugador y por instante. Ese vector se utiliza para estimar la zona mas probable del jugador, interpolar una posicion visual aproximada entre zonas y acumular la permanencia para producir un mapa de calor. El sistema no busca coordenadas exactas; busca una representacion util, estable y defendible del comportamiento espacial de cada jugador.

- **Resultado esperado del MVP:** un jugador, cuatro torres, nueve zonas, CSV de lecturas RSSI, modelo KNN y heatmap 3x3.
- **Resultado esperado de la version intermedia:** posicion visual aproximada por interpolacion de probabilidades y movimiento suavizado en pantalla.
- **Resultado esperado de la version final:** hasta 10 jugadores, 8 a 10 torres, mapa por jugador, mapa global y analisis posterior de datos.

2. Vision y alcance tecnico

2.1 Problema a resolver

En deportes, juegos tacticos, entrenamientos o actividades de campo, puede ser util saber en que regiones permanecen mas los jugadores. Las soluciones profesionales de posicionamiento pueden ser costosas o requerir infraestructura especializada. Este proyecto plantea una alternativa experimental con ESP32, enfocada en presencia por zonas y mapas de calor, no en tracking exacto de alta precision.

2.2 Objetivo general

Disenar e implementar un sistema experimental capaz de estimar la permanencia y movimiento aproximado de uno o varios ESP32 moviles dentro de un area despejada, utilizando nodos ESP32 fijos, mediciones RSSI por ESP-NOW y modelos de clasificacion/interpolacion en Python.

2.3 Objetivos especificos

14. Implementar comunicacion ESP-NOW entre nodos moviles y nodos fijos.
15. Registrar lecturas RSSI asociadas a jugador, paquete, torre y tiempo.
16. Construir un dataset etiquetado por zonas mediante calibracion en campo.
17. Entrenar un modelo supervisado que prediga el cuadrante mas probable.
18. Convertir probabilidades de zona en una posicion visual aproximada.
19. Aplicar suavizado temporal para evitar saltos bruscos en pantalla.
20. Generar mapas de calor por jugador y mapas agregados del juego.
21. Escalar el prototipo desde 1 jugador hasta 10 jugadores.

2.4 Alcance y limites

- **Incluido:** estimacion por zonas, interpolacion visual, heatmaps, analisis de datos, pruebas al aire libre y escalamiento progresivo.
- **No incluido en el MVP:** coordenadas exactas, navegacion autonoma, uso de GPS, tracking centimetro a centimetro, videoanalitica o triangulacion profesional.
- **Precision esperada:** suficiente para saber donde estuvo mas un jugador y para dibujar una posicion aproximada creible, no para medir desplazamientos exactos.
- **Entorno inicial:** zona despejada al aire libre, con torres fijas alrededor del area.

3. Arquitectura propuesta

La arquitectura recomendada separa claramente tres responsabilidades: los jugadores transmiten, las torres escuchan y el software central interpreta los datos. Esta separacion evita que los ESP32 moviles tengan que calcular su posicion y reduce el consumo computacional en los dispositivos que lleva cada jugador.

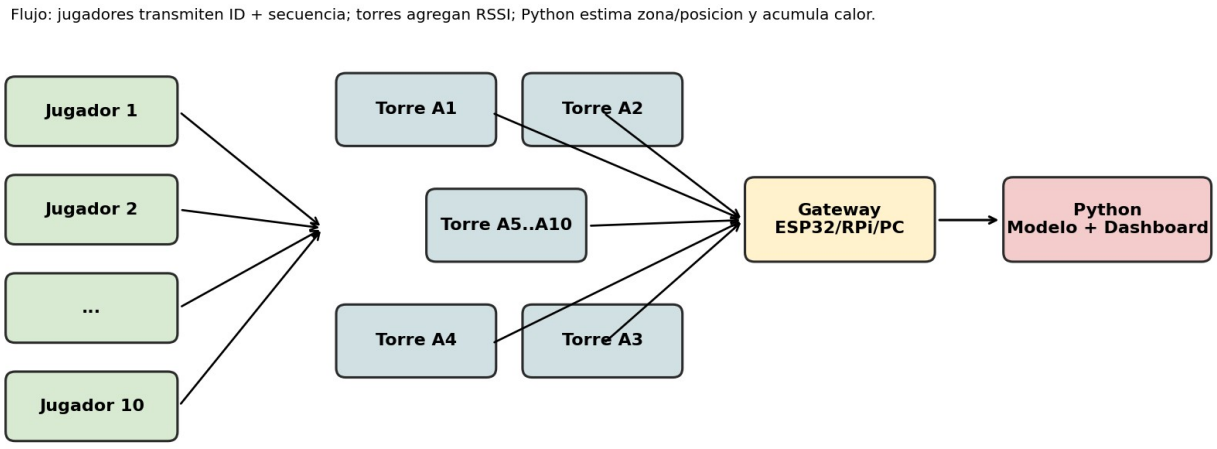


Figura 2. Flujo de informacion desde jugadores hasta el analisis en Python.

Componente	Responsabilidad	Datos que maneja
ESP32 jugador	Transmitir paquetes periodicos mediante	player_id, seq

Componente	Responsabilidad	Datos que maneja
	ESP-NOW.	
ESP32 torre	Recibir paquetes, leer RSSI y etiquetar con su anchor_id.	timestamp, player_id, seq, anchor_id, rssi
Gateway	Concentrar datos de torres y pasarlos a PC o servidor.	registros de lectura
Python	Agrupar, limpiar, predecir zona, interpolar posicion, graficar heatmap.	dataset, modelo, posiciones, mapas

Decisión clave
Los jugadores no calculan su posicion. Solo transmiten su identidad. Las torres y Python hacen el trabajo de medicion, agrupacion y estimacion.

4. Supuestos tecnicos y base de diseno

ESP-NOW es adecuado para este proyecto porque permite comunicacion directa entre dispositivos ESP32 sin depender de un router. Segun la documentacion de Espressif para ESP-IDF, ESP-NOW es un protocolo Wi-Fi connectionless en el que los datos se encapsulan en tramas vendor-specific y se transmiten entre dispositivos sin establecer una conexion tradicional. La misma documentacion indica que debe cuidarse el canal Wi-Fi, el tamano de paquete y el trabajo realizado dentro de callbacks de envio/recepcion.

- **Canal Wi-Fi:** todos los nodos deben trabajar en el mismo canal para comunicacion estable.
- **Tamano de paquete:** el proyecto debe usar paquetes pequenos, idealmente muy por debajo de 250 bytes para compatibilidad amplia.
- **Peers:** ESP-NOW tiene limites de pares registrados; se debe disenar con broadcast o con una topologia que no exceda esos limites.
- **Callbacks:** no se debe hacer trabajo pesado dentro del callback de recepcion; conviene copiar la lectura a una cola o buffer y procesarla fuera.
- **RSSI:** el RSSI debe tratarse como una senal ruidosa y relativa, no como una medida exacta de distancia.

Sobre el dato de bateria
La bateria no es necesaria para localizar. Puede enviarse opcionalmente si el ESP32 del jugador usa LiPo o power bank y se desea monitorear energia. Para el MVP, el paquete del jugador puede contener solo player_id y seq.

5. Decisiones de diseno

Decision	Eleccion recomendada	Justificacion
Metodo de ubicacion	Fingerprinting por zonas	Mas robusto que convertir RSSI a distancia exacta. Aprende patrones reales del campo.
Salida visual	Interpolacion entre centros de zonas	Permite dibujar un punto aproximado sin afirmar precision exacta.
Primer modelo	KNN	Simple, explicable y adecuado para patrones RSSI iniciales.
Modelo alternativo	Random Forest	Puede mejorar robustez y manejar relaciones no lineales.
Visualizacion	Python	Facilita pruebas, graficas, matrices de confusion y dashboard.
Frecuencia inicial	2 paquetes por segundo por jugador	Suficiente para heatmap y reduce colisiones.
Torres iniciales	4 torres	Permite validar concepto con esquinas del area.
Torres finales	8 a 10 torres	Mejor cobertura y mas informacion para el modelo.

6. Diseno espacial inicial

El MVP debe usar una cuadricula 3x3. Esta resolucion es suficientemente simple para calibrar y suficientemente rica para demostrar el concepto. Si el sistema logra distinguir nueve zonas con estabilidad, luego puede escalarse a 4x4 o 5x5.

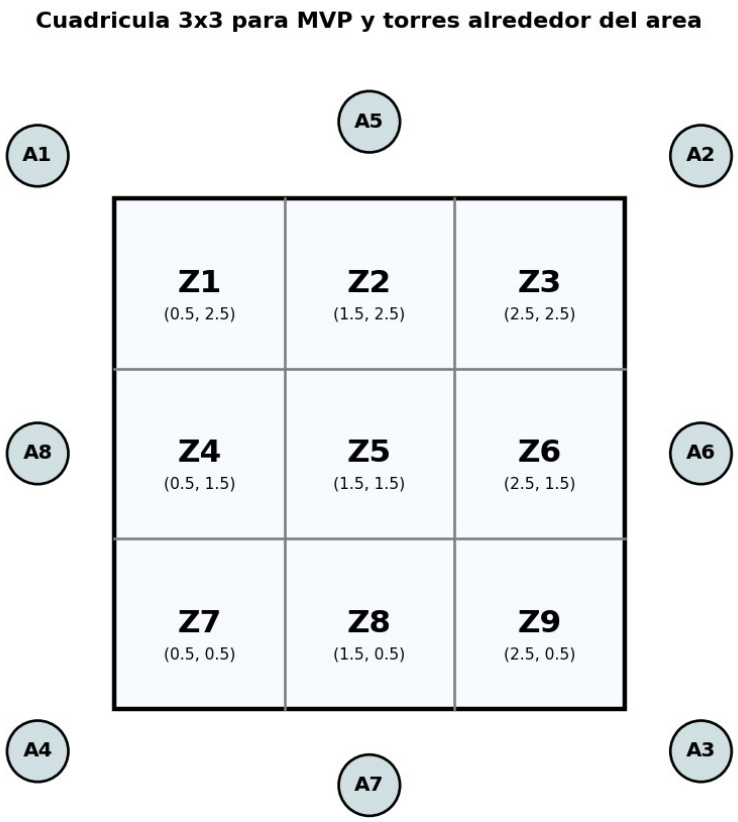


Figura 3. Cuadricula 3x3 con torres de referencia alrededor del campo.

Version	Area sugerida	Torres	Zonas	Proposito
MVP	6 m x 6 m o 10 m x 10 m	4	3x3 = 9	Validar patrones RSSI y clasificacion por zona.
Intermedia	10 m x 10 m o mayor	8	3x3 o 4x4	Mejorar interpolacion y estabilidad visual.
Final	Area real del juego	8 a 10	4x4 o 5x5	Soportar 10 jugadores y mapas individuales/globales.

7. Roadmap detallado por fases

El desarrollo debe avanzar con gates tecnicos. Cada fase produce un entregable verificable. No conviene escalar a 10 jugadores hasta que el MVP demuestre que los patrones RSSI diferencian zonas de forma consistente.

Fase	Nombre	Objetivo principal	Entregable
0	Definicion	Fijar alcance, area, zonas,	Documento de diseno inicial.

Fase	Nombre	Objetivo principal	Entregable
		torres y criterios.	
1	ESP-NOW basico	Jugador transmite y torre recibe.	RSSI visible por serial.
2	Registro	Guardar lecturas de forma estructurada.	CSV confiable.
3	Calibracion	Recolectar datos etiquetados por zona.	Dataset de entrenamiento.
4	Clasificacion	Predecir zona desde RSSI.	Modelo KNN/Random Forest.
5	Interpolacion	Dibujar posicion aproximada entre zonas.	x,y visual por jugador.
6	Suavizado	Evitar saltos bruscos en pantalla.	Movimiento estable.
7	Heatmap	Acumular permanencia por zona o coordenada.	Mapa de calor.
8	Escalamiento	Llegar a 10 jugadores y mas torres.	Sistema multijugador.

Fase 0 - Definicion del alcance

Objetivo: Convertir la idea en especificaciones medibles antes de programar.

Actividades principales

- Medir o definir el area inicial de pruebas.
- Elegir numero de torres para MVP: 4.
- Elegir cuadricula inicial: 3x3.
- Definir la frecuencia de transmision inicial: 2 Hz por jugador.
- Definir si los datos se guardan en microSD o se envian a gateway.
- Definir criterios de exito: precision por zona, latencia y estabilidad visual.

Entregables

- Diagrama de arquitectura.
- Plano del campo de prueba.
- Lista de hardware.
- Formato de datos CSV.

Criterios de exito

- Area y zonas identificadas fisicamente.
- Torre y jugador tienen ID asignado.
- Existe un protocolo de prueba escrito.

Riesgos o notas

- Cambiar el area despues de calibrar obligaria a recolectar datos nuevamente.

Fase 1 - Comunicacion ESP-NOW basica

Objetivo: Demostrar que un ESP32 jugador puede enviar paquetes y que una torre puede recibirlos con RSSI.

Actividades principales

- Configurar ambos ESP32 en el mismo canal Wi-Fi.
- Crear paquete minimo: player_id y seq.
- Transmitir por broadcast o unicast segun el prototipo.
- Registrar el callback de recepcion en la torre.
- Extraer RSSI de la informacion de recepcion disponible en el entorno usado.
- Mostrar por serial: tiempo, jugador, secuencia y RSSI.

Entregables

- Firmware del jugador.
- Firmware de una torre.
- Lecturas RSSI en monitor serial.

Criterios de exito

- Recepcion estable a diferentes distancias.
- El numero de secuencia aumenta correctamente.
- Se detectan paquetes perdidos o duplicados.

Riesgos o notas

- Usar APIs distintas entre Arduino core y ESP-IDF puede cambiar la forma de acceder a RSSI.

Fase 2 - Registro de datos

Objetivo: Guardar lecturas RSSI de forma consistente para poder analizarlas en Python.

Actividades principales

- Definir el registro normalizado: timestamp, player_id, seq, anchor_id, rssi.
- Probar guardado por microSD o envio a gateway.
- Sincronizar de forma aproximada los tiempos o agrupar por secuencia.
- Validar que cada torre usa un anchor_id unico.
- Crear scripts de importacion CSV en Python.

Entregables

- CSV por torre o CSV centralizado.
- Script Python de carga y limpieza.
- Resumen de paquetes recibidos por torre.

Criterios de exito

- Los archivos pueden unirse por player_id y seq.
- No hay columnas inconsistentes.
- Se puede calcular porcentaje de paquetes recibidos.

Riesgos o notas

- Sin identificador de secuencia, sera dificil agrupar lecturas del mismo paquete.

Fase 3 - Calibracion por zonas

Objetivo: Crear un dataset etiquetado que relacione patrones RSSI con zonas reales.

Actividades principales

- Marcar fisicamente Z1 a Z9 en el campo.
- Colocar el ESP32 movil en el centro de cada zona durante 60 segundos.
- Repetir cada zona con varias orientaciones del dispositivo.
- Registrar condiciones de prueba: altura, ubicacion en el cuerpo, hora y clima basico.
- Unir lecturas por paquete para formar vectores RSSI por instante.
- Etiquetar cada vector con la zona real.

Entregables

- Dataset etiquetado por zona.
- Bitacora de calibracion.
- Primer analisis exploratorio de RSSI por torre.

Criterios de exito

- Cada zona tiene suficientes muestras.
- El dataset tiene lecturas de todas las torres o valores faltantes controlados.
- Las zonas cercanas muestran patrones distinguibles.

Riesgos o notas

- Calibrar solo con el ESP quieto puede no representar movimiento real; conviene agregar recorridos luego.

Fase 4 - Modelo de clasificacion

Objetivo: Entrenar un modelo que prediga la zona mas probable desde un vector RSSI.

Actividades principales

- Limpiar valores faltantes: imputacion por RSSI muy bajo o ventana temporal.
- Normalizar o escalar si el modelo lo requiere.
- Entrenar KNN como linea base.
- Comparar Random Forest como modelo alternativo.
- Generar matriz de confusion y precision por zona.
- Validar con datos de una sesion distinta a la calibracion.

Entregables

- Modelo entrenado.
- Reporte de metricas.
- Matriz de confusion.
- Script reproducible de entrenamiento.

Criterios de exito

- Precision general util para el MVP.
- Zonas adyacentes pueden confundirse, pero no debe haber errores extremos frecuentes.
- El modelo es reproducible con un dataset guardado.

Riesgos o notas

- Si se entrena y evalua con datos demasiado parecidos, la precision reportada sera artificialmente alta.

Fase 5 - Posicion visual aproximada

Objetivo: Transformar probabilidades de zona en una coordenada visual para animar al jugador en pantalla.

Actividades principales

- Usar predict_proba o metodo equivalente para obtener probabilidades por zona.
- Asignar un centro x,y a cada zona.
- Calcular x,y como promedio ponderado de centros por probabilidad.
- Limitar la interpolacion a las zonas mas probables si el resultado se vuelve inestable.
- Dibujar el punto sobre el mapa de campo.

Entregables

- Funcion de interpolacion.
- x,y visual por lectura.
- Primer mapa con punto movil.

Criterios de exito

- El punto se desplaza de forma coherente hacia zonas probables.
- No se afirma que x,y sea posicion exacta.

- La salida visual es entendible para un usuario.

Riesgos o notas

- Probabilidades mal calibradas pueden colocar el punto en regiones poco realistas; se corrige con filtros y suavizado.

Fase 6 - Suavizado temporal

Objetivo: Evitar que la posicion visual salte abruptamente por ruido RSSI.

Actividades principales

- Aplicar media movil exponencial a x,y.
- Probar factores de suavizado: 0.2, 0.3, 0.5.
- Medir el compromiso entre estabilidad y retraso visual.
- Agregar filtro contra saltos imposibles si una prediccion aislada se aleja demasiado.
- Mantener posicion independiente por jugador.

Entregables

- Punto movil estable.
- Funcion de suavizado parametrizable.
- Comparacion con/sin suavizado.

Criterios de exito

- El movimiento se ve natural.
- La latencia visual sigue siendo aceptable.
- Los saltos falsos disminuyen.

Riesgos o notas

- Demasiado suavizado puede ocultar cambios reales de posicion.

Fase 7 - Generacion de mapa de calor

Objetivo: Acumular permanencia y crear visualizaciones del recorrido.

Actividades principales

- Crear heatmap por zonas: tiempo acumulado por Z1..Z9.
- Crear heatmap continuo: acumular calor en una matriz x,y.
- Separar heatmap individual por jugador.
- Crear heatmap global del juego.
- Exportar resultados a PNG, CSV y/o PDF de reporte.

Entregables

- Heatmap por zona.
- Heatmap continuo.
- Resumen por jugador.
- Graficas de permanencia.

Criterios de exito

- El heatmap coincide razonablemente con recorridos conocidos.
- Se pueden comparar jugadores.
- El resultado es interpretable sin explicar el algoritmo completo.

Riesgos o notas

- Un heatmap bonito puede ocultar errores; siempre debe acompanarse de metricas de validacion.

Fase 8 - Escalamiento a 10 jugadores

Objetivo: Extender el sistema a multiples ESP32 moviles sin perder estabilidad.

Actividades principales

- Asignar ID unico a cada jugador.
- Introducir retraso aleatorio pequeno en la transmision para reducir colisiones.
- Probar primero 2 jugadores, luego 5 y finalmente 10.
- Medir paquetes por segundo, perdida y latencia.
- Aumentar torres a 8 o 10 si el presupuesto lo permite.
- Generar posiciones y heatmaps independientes por jugador.

Entregables

- Firmware multijugador.
- Dashboard con multiples puntos.
- Mapas individuales y globales.
- Reporte de rendimiento.

Criterios de exito

- 10 jugadores transmiten sin saturar el sistema.
- Cada jugador mantiene identidad correcta.
- El heatmap individual no mezcla datos de otros jugadores.

Riesgos o notas

- A mayor numero de jugadores, aumentan colisiones y lecturas faltantes; se controla con frecuencia moderada y jitter.

8. Formato de datos recomendado

La trazabilidad del sistema depende de que cada lectura pueda relacionarse con un jugador, un paquete especifico, una torre y un tiempo. El numero de secuencia es indispensable para agrupar lecturas recibidas por varias torres.

8.1 Paquete enviado por el jugador

```
typedef struct {
    uint8_t  player_id;  // 1 a 10
    uint32_t seq;        // numero de paquete incremental
} player_packet_t;
```

8.2 Registro generado por cada torre

```
typedef struct {
    uint8_t  anchor_id;   // ID de torre: A1, A2, ...
    uint8_t  player_id;   // jugador que envio el paquete
    uint32_t seq;         // secuencia recibida
    int8_t   rssi;        // intensidad recibida en dBm
    uint32_t timestamp;   // tiempo local de recepcion
} anchor_record_t;
```

8.3 CSV minimo

```
timestamp,player_id,seq,anchor_id,rssi
15230,1,89,A1,-50
15231,1,89,A2,-61
15232,1,89,A3,-78
15233,1,89,A4,-72
```

9. Dataset, calibracion y preparacion de datos

El fingerprinting funciona porque el sistema aprende el patron de RSSI caracteristico de cada zona. La calidad del dataset es tan importante como el modelo. Una calibracion descuidada producira una visualizacion inestable aunque el codigo este bien escrito.

9.1 Protocolo de calibracion recomendado

22. Marcar las zonas fisicamente en el campo con conos, cinta o referencias visibles.
23. Colocar las torres a altura constante y registrar sus coordenadas aproximadas.
24. Ubicar el ESP32 movil en el centro de Z1 durante 60 segundos.
25. Repetir Z1 con el dispositivo orientado en cuatro direcciones.
26. Repetir el procedimiento para Z2..Z9.
27. Guardar una bitacora con fecha, hora, clima basico, altura del dispositivo y ubicacion en el cuerpo.
28. Hacer una segunda sesion de validacion con recorridos reales, no solo puntos quietos.

9.2 Construccion del vector de caracteristicas

Para cada jugador y numero de secuencia se agrupan las lecturas de las torres. El vector final contiene una columna por torre. Si una torre no recibio el paquete, se puede imputar un valor muy bajo, por ejemplo -100 dBm, o usar una estrategia de ventana temporal.

```
Entrada cruda:
seq=810, J3, A1=-62
seq=810, J3, A2=-55
seq=810, J3, A3=-70
seq=810, J3, A4=-82

Vector para el modelo:
[J3, seq=810] -> [-62, -55, -70, -82]
```

9.3 Reglas de calidad del dataset

- Todas las zonas deben tener una cantidad similar de muestras.
- Debe haber datos con distintas orientaciones del ESP32.
- Debe separarse una sesion de validacion que no se use para entrenar.
- No se debe evaluar solo con datos inmediatamente consecutivos a los de entrenamiento.
- Deben revisarse torres que casi nunca reciben paquetes o que generan RSSI anormal.

10. Modelos, interpolacion y visualizacion

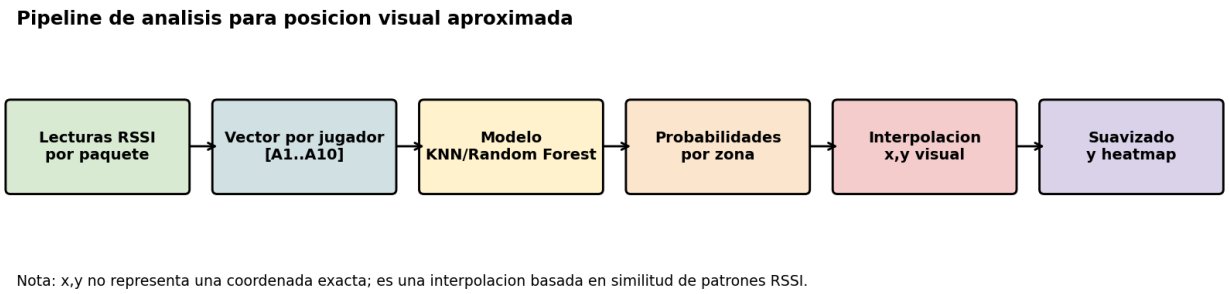


Figura 4. Pipeline recomendado para convertir RSSI en posicion visual y heatmap.

El primer modelo recomendado es KNN porque es facil de explicar: una lectura nueva se clasifica segun las muestras de calibracion mas parecidas. Random Forest es una segunda opcion si el KNN no logra estabilidad suficiente.

Modelo	Ventaja	Desventaja	Uso recomendado
KNN	Simple, explicable y funciona bien con fingerprinting.	Puede degradarse si el dataset es ruidoso o muy grande.	Linea base del MVP.
Random Forest	Robusto a ruido y relaciones no lineales.	Menos intuitivo que KNN.	Comparacion para version intermedia.
Decision Tree	Muy explicable.	Puede sobreajustar facil.	Modelo didactico o diagnostico.
KNN Regressor	Puede predecir x,y directamente.	Requiere calibracion con coordenadas mas densas.	Extension futura.

10.1 Interpolacion entre zonas

La interpolacion convierte probabilidades de zona en una posicion visual aproximada. Si el modelo indica que un jugador tiene 60% de probabilidad de estar en Z5 y 30% en Z6, el punto se dibuja entre los centros de Z5 y Z6, mas cerca de Z5.

```
x_visual = suma(prob_zona * centro_x_zona)
y_visual = suma(prob_zona * centro_y_zona)

Ejemplo:
Z5 = (50, 50), prob = 0.60
Z6 = (80, 50), prob = 0.30
Z2 = (50, 20), prob = 0.10

x = 0.60*50 + 0.30*80 + 0.10*50 = 59
y = 0.60*50 + 0.30*50 + 0.10*20 = 47
```

Frase tecnica recomendada
La coordenada x,y mostrada en pantalla no es una posicion fisica exacta; es una posicion visual interpolada a partir de probabilidades de pertenencia a zonas calibradas.

10.2 Suavizado temporal

El RSSI fluctua. Por eso, incluso si el modelo es correcto en promedio, una prediccion aislada puede mover el punto de forma brusca. Se recomienda suavizar la posicion mostrada:

```
x_display = 0.8 * x_anterior + 0.2 * x_nuevo
y_display = 0.8 * y_anterior + 0.2 * y_nuevo
```

- Un factor de 0.2 para la posicion nueva da mucho suavizado.
- Un factor de 0.5 responde mas rapido, pero puede saltar mas.
- El valor ideal se escoge mirando recorridos reales y midiendo latencia visual.

11. Mapa de calor

El mapa de calor puede generarse en dos niveles. El nivel por zonas es mas facil de validar; el continuo es mas atractivo visualmente.

Tipo	Como se calcula	Ventaja	Uso
Heatmap por zonas	Se suma tiempo a Z1..Z9 segun zona predicha.	Robusto y facil de explicar.	Reporte tecnico y validacion.
Heatmap continuo	Se suma calor alrededor de x,y visual.	Mas parecido a mapas deportivos.	Dashboard y presentacion visual.
Heatmap global	Suma todos los jugadores.	Muestra ocupacion total del juego.	Analisis estrategico.
Heatmap individual	Se calcula por player_id.	Permite comparar roles o comportamientos.	Analisis por jugador.

Para la defensa del proyecto, conviene mostrar ambos: un heatmap por zonas para demostrar rigor y un heatmap continuo para que la visualizacion se parezca a una representacion deportiva.

12. Escalamiento a 10 jugadores

El escalamiento no debe hacerse de golpe. El riesgo principal al agregar jugadores es aumentar la cantidad de paquetes simultaneos, colisiones, registros incompletos y dificultad para depurar. La escalera recomendada es 1 -> 2 -> 5 -> 10 jugadores.

Etapa	Jugadores	Torres	Frecuencia sugerida	Objetivo
Prueba A	1	4	2 Hz	Validar RSSI y zona.
Prueba B	2	4	2 Hz	Validar separacion por player_id.
Prueba C	5	4 a 8	2 Hz	Medir perdida y dashboard multiusuario.
Prueba D	10	8 a 10	1 a 2 Hz	Validar sistema final.

- **Jitter de transmision:** cada jugador puede esperar un pequeno tiempo aleatorio antes de enviar para reducir transmisiones simultaneas.
- **Identidad:** cada ESP32 jugador debe tener un player_id fijo grabado o configurable.
- **Procesamiento:** Python debe mantener buffers y suavizado separados por jugador.
- **Validacion:** cada prueba multijugador debe incluir conteo de paquetes por jugador y por torre.

13. Cronograma recomendado

El cronograma siguiente asume un desarrollo academico/prototipo con pruebas semanales. Puede comprimirse si ya se domina ESP-NOW y Python, o extenderse si se requiere fabricar carcasas, soportes o un dashboard mas elaborado.

Semana	Trabajo principal	Entregable
1	Definicion de area, hardware, IDs, cuadricula y criterios.	Documento de diseno y lista de materiales.

Semana	Trabajo principal	Entregable
2	ESP-NOW 1 jugador -> 1 torre.	RSSI por serial.
3	1 jugador -> 4 torres.	Lecturas simultaneas por torres.
4	Registro CSV o gateway.	Dataset crudo importable en Python.
5	Protocolo de calibracion y captura Z1..Z9.	Dataset etiquetado v1.
6	Limpieza, agrupacion por seq y EDA.	Graficas RSSI por zona.
7	Entrenamiento KNN y metricas.	Modelo baseline y matriz de confusion.
8	Random Forest y comparacion.	Modelo seleccionado.
9	Probabilidades e interpolacion x,y.	Punto aproximado en mapa.
10	Suavizado temporal y dashboard basico.	Movimiento estable.
11	Heatmap por zonas y continuo.	Mapas exportables.
12	Prueba con 2 a 5 jugadores.	Reporte de rendimiento parcial.
13	Prueba con 10 jugadores y 8-10 torres.	Sistema multijugador.
14	Validacion final, documentacion y presentacion.	Reporte final y demo.

14. Hardware y software recomendado

Elemento	MVP	Version final	Notas
ESP32 jugador	1	10	Uno por jugador. Idealmente mismo modelo de placa.
ESP32 torre	4	8 a 10	Colocados alrededor del campo a altura constante.
Alimentacion	Power banks o LiPo	Power banks o LiPo	Medir autonomia en pruebas largas.
Soportes	Tripodes simples	Tripodes/cajas protegidas	La altura constante mejora repetibilidad.
Gateway	Opcional	Recomendado	Puede ser ESP32 por USB, Raspberry Pi o PC.
MicroSD	Opcional	Opcional	Util si no hay comunicacion central en tiempo real.
Python	Laptop	Laptop/servidor	Pandas, scikit-learn, matplotlib/plotly.

15. Plan de pruebas y metricas

El proyecto debe evaluarse con metricas tecnicas y no solo con una demo visual. Las siguientes metricas permiten demostrar que el sistema funciona y tambien identificar limites.

Metrica	Como se mide	Interpretacion
Precision por zona	Aciertos / muestras etiquetadas.	Indica capacidad de clasificar cuadrantes.
Matriz de confusion	Comparar zona real vs zona predicha.	Muestra que zonas se confunden.
Error de zona adyacente	Errores que caen en zona vecina.	Aceptable en interpolacion; peor si salta lejos.
Perdida de paquetes	Paquetes recibidos / paquetes transmitidos.	Evalua estabilidad de red.
Latencia visual	Tiempo entre transmision y actualizacion.	Debe ser razonable para dashboard.
Jitter visual	Variacion brusca de x,y estando quieto.	Evalua necesidad de suavizado.
Consistencia de heatmap	Comparar recorrido conocido vs heatmap.	Evalua utilidad final.

Meta realista del MVP
Lograr que las zonas principales se distingan de manera estable. No es necesario alcanzar 95% de precision. Para una primera version, 70% a 85% por zona puede ser un resultado defendible si los errores son principalmente hacia zonas vecinas.

16. Riesgos principales y mitigaciones

Riesgo	Impacto	Mitigacion
RSSI ruidoso	Predicciones inestables.	Calibrar bien, usar ventanas, suavizado y mas torres.
Cuerpo bloquea antena	RSSI cambia segun orientacion.	Ubicar ESP32 consistentemente en pecho/espalda/hombro y calibrar orientaciones.
Colisiones con 10 jugadores	Paquetes perdidos.	Frecuencia moderada, jitter, payload pequeno y pruebas graduales.
Torres mal ubicadas	Zonas poco distinguibles.	Distribuir alrededor del area y evitar ubicarlas todas de un lado.
Cambio de campo	Modelo deja de representar el entorno.	Recalibrar o hacer calibraciones por escenario.
Modelos sobreajustados	Buena precision en calibracion, mala en juego real.	Separar sesiones de entrenamiento y prueba.
APIs inconsistentes	Dificultad para extraer RSSI.	Definir desde temprano si se usara ESP-IDF o Arduino core y documentarlo.

17. Checklist del MVP

- 1 ESP32 jugador transmite player_id y seq.
- 4 torres reciben paquetes y extraen RSSI.
- Cada torre tiene anchor_id unico.
- Existe CSV con timestamp, player_id, seq, anchor_id y rssi.
- Python agrupa lecturas por player_id y seq.
- Se calibra una cuadricula 3x3.
- Se entrena un modelo KNN.
- Se obtiene matriz de confusion.
- Se genera heatmap por zonas.
- Se documentan limites y precision obtenida.

18. Definicion de version final

La version final del proyecto debe demostrar un sistema multijugador capaz de mostrar posiciones visuales aproximadas y mapas de calor individuales. La descripcion recomendada para el informe o presentacion es:

Descripcion final sugerida
Sistema experimental de estimacion de presencia y movimiento aproximado de multiples jugadores en una zona despejada, mediante ESP32, ESP-NOW, RSSI, fingerprinting por cuadrantes, interpolacion de probabilidades y generacion de mapas de calor en Python.

Capacidad	MVP	Final
Jugadores	1	10
Torres	4	8 a 10
Zonas	9	16 o 25
Salida	Zona y heatmap 3x3	Posicion aproximada, heatmaps individuales y global
Tiempo real	No obligatorio	Deseable
Modelo	KNN	KNN/Random Forest comparados
Validacion	Matriz de confusion	Metricas por jugador y por zona

19. Glosario

Termino	Definicion
ESP-NOW	Protocolo de comunicacion Wi-Fi connectionless de Espressif para intercambio directo de datos entre dispositivos.
RSSI	Received Signal Strength Indicator. Indicador de intensidad de senal recibida, expresado usualmente en dBm.
Anchor / torre	ESP32 fijo que escucha paquetes y registra RSSI.
Tag / jugador	ESP32 movil portado por el jugador.
Fingerprinting	Metodo que aprende patrones de senal asociados a posiciones o zonas conocidas.
KNN	Modelo que clasifica segun las muestras mas cercanas en el espacio de caracteristicas.
Interpolacion	Metodo para convertir probabilidades por zona en una posicion visual aproximada.
Heatmap	Mapa que acumula permanencia o frecuencia de presencia en zonas del campo.

20. Referencias tecnicas

- Espressif Systems. ESP-IDF Programming Guide - ESP-NOW. https://docs.espressif.com/projects/esp-idf/en/stable/esp32/api-reference/network/esp_now.html
- Espressif Systems. ESP-FAQ - ESP-NOW. https://docs.espressif.com/projects/esp-faq/en/latest/application-solution/esp-now.html
- Espressif Systems. Arduino-ESP32 ESP-NOW API. https://docs.espressif.com/projects/arduino-esp32/en/latest/api/espnow.html
- scikit-learn documentation - Nearest Neighbors and Random Forest. https://scikit-learn.org/stable/

Nota: Las referencias tecnicas deben verificarse contra la version exacta de ESP-IDF o Arduino core usada en el proyecto, porque algunas APIs y limites pueden variar por version.